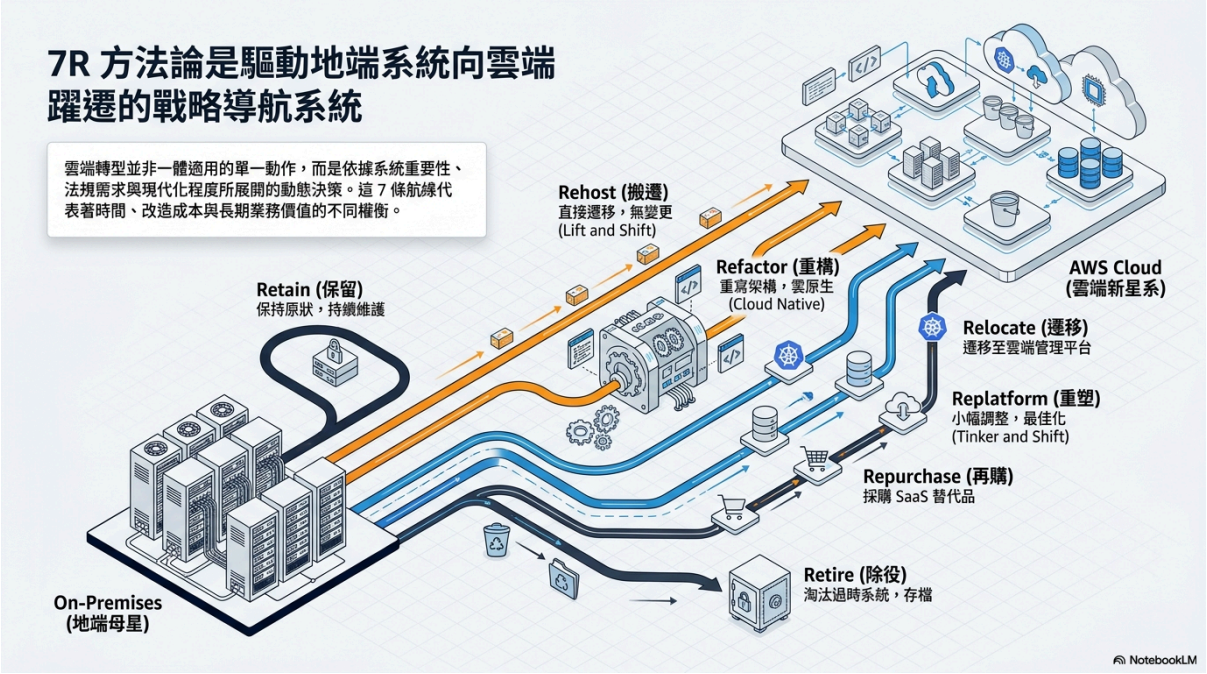
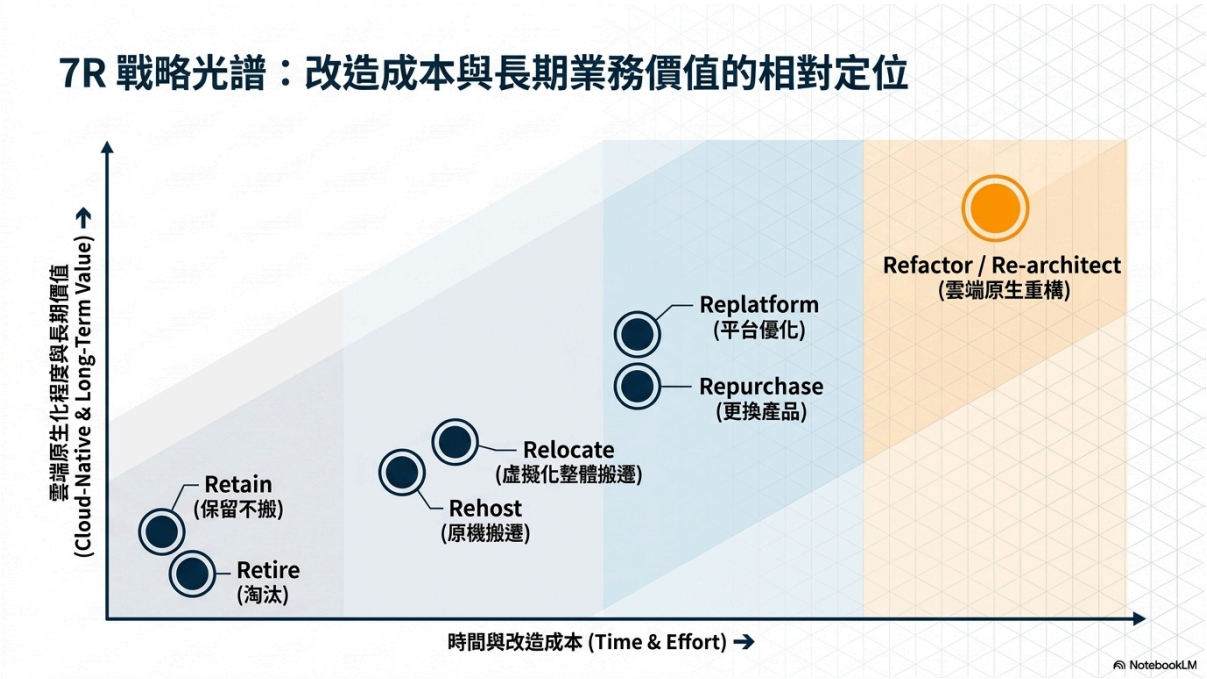
課程6 - CloudOpS

1-1 : 為什麼有託管，仍須自行監控

搬遷的要件：





許多初學者誤以為選用 AWS 託管服務後，運維責任就全部轉移給 AWS。事實上，AWS 採用「責任共同承擔模型（Shared Responsibility Model）」，AWS 負責基礎設施本身的安全性與可用性，而應用程式、資料、身份識別、日誌管理與成本控制，則完全屬於客戶自身的責任。以下四個面向說明為何「有託管仍需監控」：

1. 身份與存取管理（IAM）是你的責任：IAM Role、Policy、MFA 設定不當，AWS 不會替你防堵異常呼叫或越權操作。
2. 應用程式邏輯與錯誤：HTTP 500、Lambda 逾時、資料庫連線耗盡等問題，來自程式碼與設定，AWS 不會主動介入。
3. 成本可視性：忘記關閉閒置資源、選錯執行個體類型，都會造成費用異常。監控提供早期預警。
4. 合規與稽查：ISO 27001、SOC 2、PCI DSS 等規範要求留存 API 操作軌跡與存取記錄，必須自行設定 CloudTrail 與 Config。

1-1-1：AWS Well-Architected 架構良好框架的結合：

AWS Well-Architected Framework 提供六大支柱作為雲端系統設計的標準方法論，CloudOps 落地實踐需對應每一支柱。以下說明各支柱與監控方法的對應關係：

Well-Arch 支柱	核心監控目標	主要 AWS 服務
卓越營運 (Operational Excellence)	流程自動化、部署健康、事件回應	CloudWatch、Systems Manager、X-Ray、Cloudformation
安全性 (Security)	身份異常、資源設定偏移、威脅偵測	GuardDuty、Security Hub、CloudTrail、Config
可靠性 (Reliability)	錯誤率、重試機制、可用性 SLA	CloudWatch Alarms、Health Dashboard、Route 53
效能效率 (Performance)	延遲分析、資源瓶頸、吞吐量	CloudWatch Metrics、X-Ray Traces、Trusted Advisor
成本最佳化 (Cost Optimization)	閒置資源、成本趨勢、預算告警	Cost Explorer、Budgets、Trusted Advisor
永續性 (sustainability)	資源分層，有效利用	S3 Lifecycle等 https://www.cloudoptimo.com/blog/s3-lifecycle-policies-optimizing-cloud-storage-in-aws/

補充:AWS在資安服務上的名詞

服務	說明	使用情境
GuardDuty	智慧型威脅偵測，分析 CloudTrail、VPC Flow Logs、DNS logs	自動偵測異常 API 呼叫、可疑 IP 存取、帳號遭入侵跡象
Inspector	自動化漏洞掃描服務，針對 EC2、Lambda、容器映像檔進行 CVE 掃描	持續掃描 EC2 OS 套件漏洞、ECR 容器映像已知漏洞、Lambda 依賴套件風險
Security Hub	集中彙整多個安全服務的發現結果	統一檢視 GuardDuty、Inspector、Macie 的安全警報
CloudTrail	記錄所有 AWS API 呼叫的稽核日誌	追查「誰在何時刪除了 S3 bucket」、合規稽核
Config	持續追蹤 AWS 資源設定變更與合規狀態	確保 S3 bucket 未公開、EC2 安全群組符合規範

1-1-2:關於OSI7層與AWS服務的對應關係

在將服務移轉至雲端時，攻擊就開始了，那雲端怎麼防禦這些內容呢?

另外我們對於OSI的層級可以搭配與對應的服務參考如下:

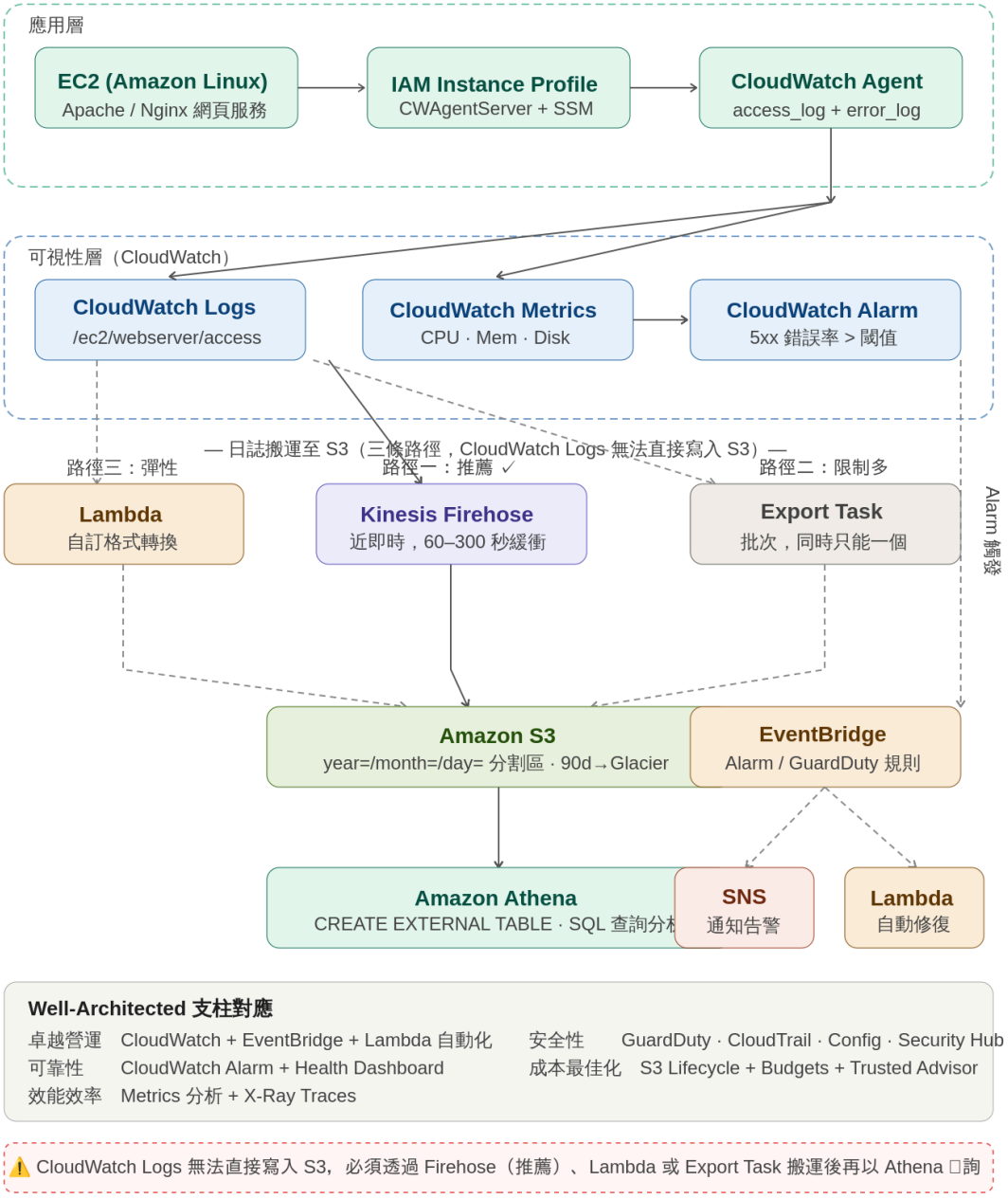


1-1-3：資安服務整合：

在 CloudOps 架構中，資安監控與一般維運監控需要同步進行。AWS 提供一套完整的原生資安服務，可與 CloudWatch 及 EventBridge 整合形成自動化的資安回應機制（SOAR）。以下為主要資安服務說明：

服務名稱	功能說明	整合方式
Amazon GuardDuty	威脅偵測服務，分析 VPC Flow Logs、DNS 查詢、CloudTrail 事件，自動識別惡意行為（如暴力破解、加密貨幣挖礦）	GuardDuty Finding → EventBridge → Lambda 自動隔離
AWS Security Hub	集中化安全態勢管理，彙整多個安全服務的結果，對應 CIS、PCI DSS 等合規標準	Security Hub → EventBridge → SNS 通知資安團隊
AWS CloudTrail	記錄所有 AWS API 呼叫，提供完整稽查軌跡，支援多帳號、多區域整合	CloudTrail → CloudWatch Logs → Alarm 設定異常 API
AWS Config	持續評估資源設定是否符合規則（如 S3 不得公開、MFA 必須啟用），偵測設定偏移	Config Rule 不合規 → EventBridge → 自動修復
Amazon Inspector	針對 EC2 與 ECR 容器映像進行漏洞掃描，輸出 CVE 報告	Inspector → Security Hub 彙整結果
AWS WAF + Shield	WAF 過濾惡意 HTTP 請求（SQL Injection、XSS）；Shield 提供 DDoS 防護	WAF Logs → Kinesis → S3 → Athena 查詢分析

1-1-4：我們來嘗試用以下的架構來輔助說明，以下是一個簡單的範例說明從前端虛擬機蒐集Log轉發到後端的服務的情境場景，進而達到事件觸發後可以收到通知。



2-1：實作環節（CloudOps Monitor）

實作一：本實作將以一台 EC2（Amazon Linux 2023）執行 Apache Web 服務為基礎，完整實踐以下 CloudOps 目標：

1. 安裝並設定 CloudWatch Agent，將 Apache access log 與 error log 推送至 CloudWatch Logs
2. 設定 Subscription Filter 將日誌轉送至 Amazon S3
3. 使用 Amazon Athena 建立資料表並以 SQL 查詢分析存取記錄
4. 設定 CloudWatch Alarm 於錯誤率超過閾值時觸發 SNS 告警

以下 CloudFormation 範本將一次建立：VPC + Subnet + IGW、EC2（含 IAM Role 與 CloudWatch Agent 啟動腳本）、S3 Log 儲存桶、CloudWatch Log Group、以及 SNS 告警主題。請將以下內容存為 cloudops-lab.yaml，並透過 AWS 主控台或 CLI 部署。

```
AWSTemplateFormatVersion: '2010-09-09'
Description: CloudOps Lab - EC2 + CloudWatch Agent + S3 + Athena

Parameters:
  KeyName:
    Type: AWS::EC2::KeyPair::KeyName
    Description: EC2 SSH Key Pair
  YourIP:
    Type: String
    Default: '0.0.0.0/0'
    Description: Your IP CIDR for SSH access (e.g. 1.2.3.4/32)

Resources:

  # ── VPC 網路 ──────────────────────────────────
  CloudOpsVPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: 10.0.0.0/16
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags: [{Key: Name, Value: cloudops-vpc}]

  PublicSubnet:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref CloudOpsVPC
      CidrBlock: 10.0.1.0/24
      MapPublicIpOnLaunch: true
      AvailabilityZone: !Select [0, !GetAZs '']
      Tags: [{Key: Name, Value: cloudops-public-subnet}]

  IGW:
    Type: AWS::EC2::InternetGateway

  IGWAttach:
    Type: AWS::EC2::VPCGatewayAttachment
```

```
  Properties:
    VpcId: !Ref CloudOpsVPC
    InternetGatewayId: !Ref IGW

RouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref CloudOpsVPC

DefaultRoute:
  Type: AWS::EC2::Route
  DependsOn: IGWAttach
  Properties:
    RouteTableId: !Ref RouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref IGW

SubnetRouteAssoc:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet
    RouteTableId: !Ref RouteTable

# ── Security Group ──────────────────────────────
WebSG:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: Web + SSH
    VpcId: !Ref CloudOpsVPC
    SecurityGroupIngress:
      - IpProtocol: tcp
        FromPort: 80
        ToPort: 80
        CidrIp: 0.0.0.0/0
      - IpProtocol: tcp
        FromPort: 443
        ToPort: 443
        CidrIp: 0.0.0.0/0
      - IpProtocol: tcp
        FromPort: 22
        ToPort: 22
        CidrIp: !Ref YourIP

# ── IAM Role for EC2 ────────────────────────────
EC2Role:
  Type: AWS::IAM::Role
  Properties:
    RoleName: cloudops-ec2-role
    AssumeRolePolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: Allow
          Principal: {Service: ec2.amazonaws.com}
          Action: sts:AssumeRole
    ManagedPolicyArns:
      - arn:aws:iam::aws:policy/CloudWatchAgentServerPolicy
      - arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore

EC2InstanceProfile:
```

```
  Type: AWS::IAM::InstanceProfile
  Properties:
    Roles: [!Ref EC2Role]

# ── S3 Bucket for Logs ──────────────────────────
LogBucket:
  Type: AWS::S3::Bucket
  Properties:
    BucketName: !Sub 'cloudops-logs-${AWS::AccountId}-${AWS::Region}'
    LifecycleConfiguration:
      Rules:
        - Id: archive-after-90d
          Status: Enabled
          Transitions:
            - TransitionInDays: 90
              StorageClass: GLACIER

# ── CloudWatch Log Groups ──────────────────────
AccessLogGroup:
  Type: AWS::Logs::LogGroup
  Properties:
    LogGroupName: /ec2/webserver/access
    RetentionInDays: 90

ErrorLogGroup:
  Type: AWS::Logs::LogGroup
  Properties:
    LogGroupName: /ec2/webserver/error
    RetentionInDays: 90

# ── SNS Topic ──────────────────────────────────
AlarmTopic:
  Type: AWS::SNS::Topic
  Properties:
    TopicName: cloudops-alarm
    DisplayName: CloudOps Alarm

# ── CloudWatch Alarm: HTTP 5xx rate ────────────
Error5xxAlarm:
  Type: AWS::CloudWatch::Alarm
  Properties:
    AlarmName: High-5xx-ErrorRate
    AlarmDescription: HTTP 5xx errors exceeded threshold
    MetricName: ErrorCount
    Namespace: CloudOpsLab/WebServer
    Statistic: Sum
    Period: 300
    EvaluationPeriods: 1
    Threshold: 10
    ComparisonOperator: GreaterThanThreshold
    TreatMissingData: notBreaching
    AlarmActions: [!Ref AlarmTopic]

# ── EC2 Instance ───────────────────────────────
WebEC2:
  Type: AWS::EC2::Instance
  Properties:
    ImageId: ami-0f3a440bbcff3d043  # Amazon Linux 2023 ap-northeast-1
    InstanceType: t3.micro
```

```
      KeyName: !Ref KeyName
      IamInstanceProfile: !Ref EC2InstanceProfile
      NetworkInterfaces:
        - AssociatePublicIpAddress: true
          DeviceIndex: 0
          SubnetId: !Ref PublicSubnet
          GroupSet: [!Ref WebSG]
      UserData:
        Fn::Base64: !Sub |
          #!/bin/bash
          dnf update -y
          dnf install -y httpd amazon-cloudwatch-agent
          systemctl enable --now httpd
          echo '<h1>CloudOps Lab</h1>' > /var/www/html/index.html
          # ── CloudWatch Agent 設定 ──────────────────
          cat > /opt/aws/amazon-cloudwatch-agent/etc/cloudwatch-config.json
<< 'CWEOF'
          {
            "logs": {
              "logs_collected": {
                "files": {
                  "collect_list": [
                    {
                      "file_path": "/var/log/httpd/access_log",
                      "log_group_name": "/ec2/webserver/access",
                      "log_stream_name": "{instance_id}/access",
                      "timestamp_format": "%d/%b/%Y:%H:%M:%S %z"
                    },
                    {
                      "file_path": "/var/log/httpd/error_log",
                      "log_group_name": "/ec2/webserver/error",
                      "log_stream_name": "{instance_id}/error",
                      "timestamp_format": "%a %b %d %H:%M:%S.%f %Y"
                    }
                  ]
                }
              },
              "metrics": {
                "namespace": "CloudOpsLab/WebServer",
                "metrics_collected": {
                  "cpu": {"measurement": ["usage_idle","usage_user"]},
                  "mem": {"measurement": ["mem_used_percent"]},
                  "disk": {"measurement": ["used_percent"]}
                }
              }
            }
          CWEOF
          /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
            -a fetch-config -m ec2 \
            -c
file:/opt/aws/amazon-cloudwatch-agent/etc/cloudwatch-config.json -s

Outputs:
  WebURL:
    Value: !Sub 'http://${WebEC2.PublicIp}'
  LogBucketName:
    Value: !Ref LogBucket
  AlarmTopicArn:
```

```
      Value: !Ref AlarmTopic
```

2-1-1：手動安裝與設定 CloudWatch Agent（Amazon Linux）

若未使用 CloudFormation UserData 自動安裝，可依以下步驟手動完成 CloudWatch Agent 的安裝與設定。

步驟一：確認 **IAM Instance Profile** 已附加正確 **Policy**

EC2 執行個體的 IAM Role 需要具備以下受管政策（Managed Policy）：

CloudWatchAgentServerPolicy（推送指標與日誌）
AmazonSSMManagedInstanceCore（透過 Systems Manager 遠端管理）

步驟二：安裝 **CloudWatch Agent**

```
# Amazon Linux 2023
sudo dnf install -y amazon-cloudwatch-agent

# Amazon Linux 2 (舊版)
sudo yum install -y amazon-cloudwatch-agent
```

步驟三：建立 **Agent** 設定檔

將以下內容存為 /opt/aws/amazon-cloudwatch-agent/etc/cloudwatch-config.json：

```
{
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/httpd/access_log",
            "log_group_name": "/ec2/webserver/access",
            "log_stream_name": "{instance_id}/access",
            "timestamp_format": "%d/%b/%Y:%H:%M:%S %z",
            "multi_line_start_pattern": "{timestamp_format}"
          },
          {
            "file_path": "/var/log/httpd/error_log",
            "log_group_name": "/ec2/webserver/error",
            "log_stream_name": "{instance_id}/error",
            "timestamp_format": "%a %b %d %H:%M:%S.%f %Y"
          }
        ]
      }
    }
  },
  "metrics": {
    "namespace": "CloudOpsLab/WebServer",
    "metrics_collected": {
      "cpu": { "measurement": ["usage_idle", "usage_user", "usage_system"]
},
      "mem": { "measurement": ["mem_used_percent"] },
      "disk": { "measurement": ["used_percent"], "resources": ["/"] }
    },
```

```
    "append_dimensions": {
      "InstanceId": "${aws:InstanceId}"
    }
  }
}
```

步驟四：啟動 CloudWatch Agent

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config -m ec2 \
  -c file:/opt/aws/amazon-cloudwatch-agent/etc/cloudwatch-config.json -s

# 確認服務狀態
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a
status

# 預期輸出：
# "status": "running"
```

步驟五：驗證日誌已上傳至 CloudWatch Logs

```
# 透過 AWS CLI 確認 Log Group 已建立
aws logs describe-log-groups --log-group-name-prefix '/ec2/webserver'

# 查看最新 Log Streams
aws logs describe-log-streams \
  --log-group-name '/ec2/webserver/access' \
  --order-by LastEventTime --descending

# 模擬產生 access log（在 EC2 上執行）
for i in $(seq 1 20); do curl http://localhost/; done
```

2-1-2：將 EC2 網頁日誌導入 Amazon Athena 查詢

Amazon Athena 是 Serverless 的互動式查詢服務，直接對 S3 中的日誌資料執行 SQL，無需建立資料庫或管理伺服器，非常適合用於日誌分析。整體流程分為：CloudWatch Logs → Kinesis Firehose → S3 → Athena。

步驟一：設定 **CloudWatch Logs Subscription Filter** 至 **Kinesis Firehose**

在 AWS 主控台 CloudWatch Logs → Log Group → Subscription Filter，選擇 Kinesis Firehose 作為目的地。

或透過 CLI 設定：

```
# 建立 Firehose Delivery Stream（目的地為 S3）
aws firehose create-delivery-stream \
  --delivery-stream-name cloudops-access-logs \
  --s3-destination-configuration \
    RoleARN=arn:aws:iam::ACCOUNT_ID:role/firehose-role,\
    BucketARN=arn:aws:s3:::cloudops-logs-ACCOUNT_ID-REGION,\

Prefix=webserver/access/year=!{timestamp:yyyy}/month=!{timestamp:MM}/day=!{t
imestamp:dd}/

# 建立 Subscription Filter
aws logs put-subscription-filter \
  --log-group-name '/ec2/webserver/access' \
  --filter-name 'all-events' \
  --filter-pattern '' \
  --destination-arn
arn:aws:firehose:REGION:ACCOUNT_ID:deliverystream/cloudops-access-logs
```

步驟二：在 **Athena** 建立資料庫與資料表

進入 Amazon Athena 主控台，選擇 Query Editor，依序執行以下 SQL：

```
-- 建立資料庫
CREATE DATABASE IF NOT EXISTS cloudops_logs;

-- 建立 Apache CLF Access Log 資料表
CREATE EXTERNAL TABLE IF NOT EXISTS cloudops_logs.apache_access (
  host          STRING,
  identity      STRING,
  user          STRING,
  request_time  STRING,
  request       STRING,
  status        INT,
  bytes         INT,
  referrer      STRING,
  user_agent    STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.RegexSerDe'
WITH SERDEPROPERTIES (
```

```
  'serialization.format' = '1',
  'input.regex' = '([^ ]*) ([^ ]*) ([^ ]*) \\[([^]]*)\\] "([^"]*)" ([0-9]*)
([0-9-]*) "([^"]*)" "([^"]*)"'
)
LOCATION 's3://cloudops-logs-ACCOUNT_ID-REGION/webserver/access/'
TBLPROPERTIES ('has_encrypted_data'='false');
```

步驟三：執行常用查詢範例

```
-- 查詢最近 100 筆存取記錄
SELECT * FROM cloudops_logs.apache_access LIMIT 100;

-- 統計各 HTTP 狀態碼數量
SELECT status, COUNT(*) AS count
FROM cloudops_logs.apache_access
GROUP BY status ORDER BY count DESC;

-- 找出 HTTP 5xx 錯誤的請求
SELECT request_time, host, request, status
FROM cloudops_logs.apache_access
WHERE status >= 500
ORDER BY request_time DESC LIMIT 50;

-- 統計最多存取的前 10 個 URL
SELECT
  SPLIT_PART(request, ' ', 2) AS path,
  COUNT(*) AS hits
FROM cloudops_logs.apache_access
GROUP BY 1 ORDER BY hits DESC LIMIT 10;

-- 使用分割區（Partition）加速查詢（依日期）
SELECT * FROM cloudops_logs.apache_access
WHERE year='2025' AND month='07' AND day='01'
LIMIT 100;
```

步驟四：設定 **Athena** 查詢結果儲存位置

在 Athena 主控台 → Settings → Query result location，設定為 s3://cloudops-logs-ACCOUNT_ID-REGION/athena-results/。建議同時啟用 Amazon S3 Server-Side Encryption（SSE-S3）保護查詢結果。